Errata for "Balanced Product Quantum Codes"

Identified via MerLean: Lean 4 + Mathlib formalization cross-check

March 2026

Abstract

This document catalogues discrepancies between the paper *Balanced Product Quantum Codes* and its Lean 4 + Mathlib formalization produced by the MerLean.

Contents

Part I: Substantive Corrections **2**

0.1 Erratum S1: Incorrect α', b, and β'' in Theorem 8 . . . 2

0.2 Erratum S2: Missing β_{co} factor in D_X bound of Corollary 23 . . . 3

Part II: Minor Corrections **5**

0.3 Erratum M1: Wrong index in coboundary operator definition . . . 5

0.4 Erratum M2: Reversed matrix order in CSS commutativity condition . . . 6

0.5 Erratum M3: Swapped vertical and horizontal differentials in tensor product double complex . . . 7

0.6 Erratum M4: Augmentation symbol ϵ conflated with induced map $\bar{\epsilon}$ on $H_0(F)$ . . . 8

0.7 Erratum M5: Strict vs. non-strict inequality in Cheeger constant definition . . . 9

0.8 Erratum M6: "Convex" should be "concave" in the proof of edge-to-vertex expansion 10

0.9 Erratum M7: Alon–Chung contrapositive requires strict inequality . . . 11

0.10 Erratum M8: Strict positivity of b required in relative vertex-to-edge expansion . . . 12

0.11 Erratum M9: Incorrect use of $d_{L^\perp}$ instead of $d_{L^\perp} - 1$ in the formula for β in Theorem 9 13

0.12 Erratum M10: Strict vs. non-strict inequality in the Ramanujan bound . . . 15

0.13 Erratum M11: Missing $\frac{1}{|H|}$ normalization in balanced product isomorphism . . . 16

0.14 Erratum M12: Swapped left/right H-action convention in Künneth balanced product lemma . . . 17

0.15 Erratum M13: Incorrect orbit membership in quotient graph adjacency definition . . 19

0.16 Erratum M14: Incorrect index range for cycle graph chain groups . . . 20

Concordance Table **22**

Part I: Substantive Corrections

Confirmed formula and derivation errors

0.1 Erratum S1: Incorrect α', b, and β'' in Theorem 8

Location. Section III-C, Theorem 8 and its proof, lines 239–259

Severity. High — the stated formula for β'' is wrong in both numerator and denominator, affecting the quantitative expansion bound and any downstream distance estimates that depend on it

Description. The proof of Theorem 8 contains three coupled errors. First, the handshaking identity $2|X_1| = s|X_0|$ gives $2\alpha|X_1| = \alpha s|X_0|$, not $\frac{4\alpha}{s}|X_0|$ as claimed, so the effective vertex-fraction parameter should be $\alpha' = \alpha s$ rather than $\frac{4\alpha}{s}$. Second, Lemma 3 is applied with $b = d_L$, yielding $A = \{v \in S \mid |(\delta S)_v| \geq s - d_L\}$; but to guarantee that each vertex in A has a violated check, one needs the number of within-S edges at v to be at most $d_L - 1$ (strictly less than the local distance), which requires $b = d_L - 1$ and $A = \{v \in S \mid |(\delta S)_v| \geq s - d_L + 1\}$. These two corrections propagate into β''.

Original

$$\beta'' = \frac{(d_L - \lambda_2) - \frac{4\alpha}{s}(s - \lambda_2)}{d_L}$$

$$A = \{v \in S \mid |(\delta S)_v| \geq s - d_L\}.$$

$$|S| \leq 2|E| \leq 2\alpha|X_1| = \frac{4\alpha}{s}|X_0|$$

Corrected

$$\beta'' = \frac{(d_L - 1 - \lambda_2) - \alpha s(s - \lambda_2)}{d_L - 1}$$

$$A = \{v \in S \mid |(\delta S)_v| \geq s - (d_L - 1)\}.$$

$$|S| \leq 2|E| \leq 2\alpha|X_1| = \alpha s\,|X_0|$$

Thm_8_ExpanderViolatedChecks.lean, lines 35--41

```
/-- The vertex-to-edge-boundary expansion parameter (*$\beta''$*):
`(*$\beta''$*) = ((dL - 1 - (*$\lambda_2$*)) - (*$\alpha$*)s(s - (*$\lambda_2$
    *))) / (dL - 1)`.
This is obtained by applying Lem_3 with `b = dL - 1` and `(*$\alpha'$*) = (*$\
    alpha$*)s`.
Note: The paper's stated formula uses `4(*$\alpha$*)/s` and denominator `dL`,
    but the
proof derivation yields `(*$\alpha$*)s` (since `|S| (*$\leq$*) 2(*$\alpha$*)|X
    (*$_1$*)| = (*$\alpha$*)s|X(*$_0$*)|`) and
uses `b = dL - 1` to ensure all high-expansion vertices have violated checks.
    -/
noncomputable def beta'' (s : (*$\mathbb{R}$*)) (lam2 : (*$\mathbb{R}$*)) (
    alpha : (*$\mathbb{R}$*)) (dL : (*$\mathbb{R}$*)) : (*$\mathbb{R}$*) :=
  ((dL - 1 - lam2) - alpha * s * (s - lam2)) / (dL - 1)
```

Explanation. The correction involves three interrelated fixes, all confirmed by the Lean formalization.

Fix 1 ($\alpha' = \alpha s$, not $4\alpha/s$). The paper bounds $|S| \leq 2|E| \leq 2\alpha|X_1|$ and then converts to vertex count via $2\alpha|X_1| = \frac{4\alpha}{s}|X_0|$. However, the handshaking lemma for an s-regular graph states $2|X_1| = s|X_0|$ (see `regular_handshaking`, line 256 of the Lean file), so the correct conversion is $2\alpha|X_1| = \alpha \cdot s \cdot |X_0|$. The Lean proof establishes this chain explicitly in `incidentToSupport_card_le_alphaS` (lines 298–315), which concludes $|S| \leq \alpha s\,|X_0|$.

Fix 2 ($b = d_L - 1$, not d_L). The paper defines $A = \{v \in S \mid |(\delta S)_v| \geq s - d_L\}$, which means vertices in A can have up to d_L within-S edges. The violated-check argument requires the local view at v to be a nonzero vector of Hamming weight *strictly less than* d_L (so it cannot be a codeword of the local code). One therefore needs at most $d_L - 1$ within-S edges, i.e. $|(\delta S)_v| \geq s - (d_L - 1)$. This corresponds to applying Lemma 3 with $b = d_L - 1$. The Lean proof makes this precise: `differentialWeight_ge_highExpansion` (line 363) uses `highExpansionVertices G S s (↑dL - 1)`, and `violated_check_of_small_support` (lines 58–76) requires `hx_wt : hammingWeight x ≤ dL - 1` to derive the contradiction.

Fix 3 (corrected β''). With $\alpha' = \alpha s$ and $b = d_L - 1$ substituted into the formula of Lemma 3, $\beta = ((b - \lambda_2) - \alpha'(s - \lambda_2))\,b^{-1}$, we obtain $\beta'' = ((d_L - 1 - \lambda_2) - \alpha s(s - \lambda_2))/(d_L - 1)$, matching the Lean definition of `beta''` at line 41. The main theorem `expanderViolatedChecks` (lines 433–553) assembles these pieces and proves $|\partial(x)| \geq \beta' \cdot \beta'' \cdot |x|$ with the corrected β''.

0.2 Erratum S2: Missing β_{co} factor in D_X bound of Corollary 23

Location. Section IV-F, Corollary 23 (`cor:distanceboundssybsystemcode`), line 2133

Severity. Low — The missing β_{co} factor affects only the $\Theta(q^3)$ terms in the D_X bound; the asymptotic minimum is dominated by the $\ell\,\alpha_{\mathrm{co}}\beta_{\mathrm{co}}/(4s) = \Theta(q)$ terms, so the final result (Theorem 24) is unaffected.

Description. In the D_X bound of Corollary 23, the second term inside the minimum is written as $\alpha_{\mathrm{co}}|X_1|/2$, but it should be $\alpha_{\mathrm{co}}\beta_{\mathrm{co}}|X_1|/2$. This term arises from Theorem 22 (`thm:distco`), Case 1, which gives the bound $|X_0|\,s\min\{\alpha_{\mathrm{co}}/2,\ \alpha_{\mathrm{co}}\beta_{\mathrm{co}}/4\}$; substituting $|X_0|s = 2|X_1|$ yields $\alpha_{\mathrm{co}}|X_1|$ and $\alpha_{\mathrm{co}}\beta_{\mathrm{co}}|X_1|/2$ as the two sub-terms, but the factor β_{co} was dropped from the latter.

Original

$$D_X \geq \min\{\alpha_{\mathrm{co}}|X_1|, \alpha_{\mathrm{co}}|X_1|/2, \ell\alpha_{\mathrm{co}}/(4s), \ell\alpha_{\mathrm{co}}\beta_{\mathrm{co}}/(4s)\}$$

Corrected

$$D_X \geq \min\{\alpha_{\mathrm{co}}|X_1|, \alpha_{\mathrm{co}}\beta_{\mathrm{co}}|X_1|/2, \ell\alpha_{\mathrm{co}}/(4s), \ell\alpha_{\mathrm{co}}\beta_{\mathrm{co}}/(4s)\}$$

Thm_15_ExplicitFamilyQuantumCodes.lean, lines 262--271

```
/-- **Cohomological Distance Bound** (Panteleev--Kalachev [PK22], Theorem 14).
The X-distance of the subsystem code satisfies
`D_X >= min(|X_0|s * min(alpha_co/2, alpha_co*beta_co/4), l * min(alpha_co/(4s
    ), alpha_co*beta_co/(4s)))`.
Using `|X_0|s = 2|X_1| in Theta(q^3)` and `l = q`, the minimum is `Theta(q)`.
The expansion parameters `alpha_co = 10^{-5}`, `beta_co > 0` are from Theorem
    9. -/
axiom lps_DX_lower_bound :
  exists (c : R), 0 < c /\
    forall vp : ValidPrime,
      let D := balanced_product_construction vp
      (D.code.dX : R) >= c * (vp.q : R)
```

Explanation. The corollary collapses the two cases of Theorem 22 (`thm:distco`) into a single four-way minimum. Case 1 of Theorem 22 states that when the vertical cohomological projection is non-trivial, $|x| \geq |X_0|\,s\min\{\alpha_{\mathrm{co}}/2,\ \alpha_{\mathrm{co}}\beta_{\mathrm{co}}/4\}$. Since $|X_0|s = 2|X_1|$ (the handshaking lemma for an s-regular graph), the two sub-terms expand to $2|X_1|\cdot\alpha_{\mathrm{co}}/2 = \alpha_{\mathrm{co}}|X_1|$ and $2|X_1|\cdot\alpha_{\mathrm{co}}\beta_{\mathrm{co}}/4 = \alpha_{\mathrm{co}}\beta_{\mathrm{co}}|X_1|/2$. The paper's second term omits the factor β_{co}, writing $\alpha_{\mathrm{co}}|X_1|/2$ instead. The Lean formalization in `Thm_15_ExplicitFamilyQuantumCodes.lean` records the correct formula with $\alpha_{\mathrm{co}}\beta_{\mathrm{co}}/4$ in the docstring of the `lps_DX_lower_bound` axiom (line 263) and bounds $D_X \geq c\,q$ for a positive constant c (lines 267–271), which absorbs the product $\alpha_{\mathrm{co}}\beta_{\mathrm{co}}$ into the constant. The correction does not affect Theorem 24 because, for large q, the minimum over the four terms is dominated by the $\Theta(q)$ terms $\ell\,\alpha_{\mathrm{co}}/(4s)$ and $\ell\,\alpha_{\mathrm{co}}\beta_{\mathrm{co}}/(4s)$, not the $\Theta(q^3)$ terms involving $|X_1|$.

Part II: Minor Corrections

Notational, typographical, and convention fixes

0.3 Erratum M1: Wrong index in coboundary operator definition

Location. Section II-A ("Complexes of Vector Spaces"), `sec_complexes.tex`, line 32

Severity. Moderate — The coboundary definition $\delta^i = (\partial_{i-1})^{tr}$ has an incorrect subscript index, making the stated domain and codomain inconsistent with the transpose of ∂_{i-1}. This is a notational error that could mislead readers attempting to verify the cochain complex construction, though subsequent uses of δ in the paper are index-free and unaffected.

Description. The paper defines the coboundary operator as $\delta^i = (\partial_{i-1})^{tr} : C^i \to C^{i+1}$, but this is dimensionally inconsistent: since $\partial_{i-1} : C_{i-1} \to C_{i-2}$, its transpose maps $(C_{i-2})^* \to (C_{i-1})^*$, i.e., $C^{i-2} \to C^{i-1}$, not $C^i \to C^{i+1}$. The correct formula is $\delta^i = (\partial_{i+1})^{tr}$, since $\partial_{i+1} : C_{i+1} \to C_i$ and therefore $(\partial_{i+1})^{tr} : (C_i)^* = C^i \to (C_{i+1})^* = C^{i+1}$.

Original

We denote the transposed maps of the differential ∂ by

$$\delta^i = (\partial_{i-1})^{tr} : C^i \to C^{i+1}$$

Corrected

We denote the transposed maps of the differential ∂ by

$$\delta^i = (\partial_{i+1})^{tr} : C^i \to C^{i+1}$$

Rem_2_NotationConventions.lean, lines 61--94

```
/-- A cochain complex of F2-vector spaces indexed by Z. The differential ‘d i
    j‘
maps from degree ‘i‘ to degree ‘j‘ where ‘i + 1 = j‘. In the paper, the
    coboundary
‘delta^i : C^i -> C^{i+1}‘ is the transpose of ‘partial_{i+1}‘. -/
abbrev CochainComplexℙ₂ := CochainComplex (ModuleCat ℙ₂) ℤ

/-- The coboundary map ‘delta^i : C^i -> C^{i+1}‘ in a cochain complex is
‘C.d i j‘ in Mathlib where ‘(ComplexShape.up Z).Rel i j‘. -/
abbrev coboundary (C : CochainComplexℙ₂) (i j : ℤ) := C.d i j

/-- The transpose (dual) of a linear map ‘f : M -> N‘ is
‘Module.Dual.transpose f : N* -> M*‘. In the paper, ‘delta = partial^T‘. -/
abbrev transpose {M N : Type*} [AddCommMonoid M] [AddCommMonoid N]
    [Module ℙ₂ M] [Module ℙ₂ N] (f : M →ₗ[ℙ₂] N) := Module.Dual.transpose f
```

Explanation. The coboundary operator δ^i must map $C^i \to C^{i+1}$, so it must be the transpose of a boundary map whose codomain is C_i and whose domain is C_{i+1}. The boundary map $\partial_{i+1} : C_{i+1} \to C_i$ is exactly such a map, and its transpose $(\partial_{i+1})^{tr}$ maps $(C_i)^* = C^i$ to $(C_{i+1})^* = C^{i+1}$, as required. By contrast, $\partial_{i-1} : C_{i-1} \to C_{i-2}$ transposes to a map $C^{i-2} \to C^{i-1}$, which has nothing to do with degree i. The Lean formalization confirms the correct index: the docstring for `CochainComplexℙ₂` (line 63) explicitly states that "the coboundary $\delta^i : C^i \to C^{i+1}$ is the transpose of ∂_{i+1}," and the `transpose` abbreviation (lines 91–94) formalizes this via `Module.Dual.transpose`, which sends a linear map $f : M \to N$ to $f^{tr} : N^* \to M^*$, matching the corrected formula.

0.4 Erratum M2: Reversed matrix order in CSS commutativity condition

Location. Section II-B, "Stabilizer quantum codes," line 85 of `sec_complexes.tex`

Severity. Moderate — The displayed matrix product $H_Z^T H_X$ reverses the order required by the chain-complex composition $\partial^2 = 0$; the surrounding diagram is correct, so a careful reader can recover the intended meaning, but the formula as printed is wrong.

Description. The paper states that the CSS commutativity condition is $H_Z^T H_X = 0$. However, the chain complex displayed immediately afterward is $C_1 \xrightarrow{H_Z^T} C_0 \xrightarrow{H_X} C_{-1}$, and the $\partial^2 = 0$ condition requires the composition of consecutive differentials to vanish, giving $H_X \circ H_Z^T = 0$, i.e. $H_X H_Z^T = 0$. The paper's expression reverses the order of the matrix product.

Original

The condition that stabilizers need to commute is equivalent to demanding $H_Z^T H_X = 0$.

Corrected

The condition that stabilizers need to commute is equivalent to demanding $H_X H_Z^T = 0$.

Def_4_CSSCode.lean, lines 38--50

```
/-- A CSS (Calderbank-Shor-Steane) quantum code is defined by parity check
    matrices
`H_X : F₂^n → F₂^{r_X}` and `H_Z : F₂^n → F₂^{r_Z}` satisfying the CSS
    condition
`H_X ∘ H_Z^T = 0`. Here `H_Z^T : F₂^{r_Z} → F₂^n` is the transpose of `H_Z`,
which serves as the differential from degree 1 to degree 0 in the chain
    complex
`C₁ →[H_Z^T] C₀ →[H_X] C_{-1}`. -/
structure CSSCode (n rX rZ : ℕ) where
  /-- The X-type parity check matrix `H_X : F₂^n → F₂^{r_X}`. -/
  HX : (Fin n → F₂) →ₗ[F₂] (Fin rX → F₂)
  /-- The Z-type parity check matrix transpose `H_Z^T : F₂^{r_Z} → F₂^n`.
  This is the differential from degree 1 to degree 0 in the chain complex. -/
  HZT : (Fin rZ → F₂) →ₗ[F₂] (Fin n → F₂)
  /-- The CSS condition: `H_X ∘ H_Z^T = 0`, ensuring `∂² = 0`. -/
  css_condition : HX.comp HZT = 0
```

Explanation. In the chain complex $C_1 \xrightarrow{H_Z^T} C_0 \xrightarrow{H_X} C_{-1}$, the differential from degree 1 to degree 0 is H_Z^T and the differential from degree 0 to degree -1 is H_X. The condition $\partial^2 = 0$ requires that the composition of two consecutive differentials vanishes: $\partial_0 \circ \partial_1 = H_X \circ H_Z^T = 0$. As a matrix product this is $H_X H_Z^T = 0$, not $H_Z^T H_X = 0$. The Lean formalization encodes exactly this: the field `css_condition : HX.comp HZT = 0` asserts $H_X \circ H_Z^T = 0$, since `LinearMap.comp` applies the left argument after the right argument (i.e. `f.comp g = f ∘ g`). This is further confirmed by the proof of $\partial^2 = 0$ in the `d_comp_d'` field (lines 114–158), where the composition `ModuleCat.ofHom Q.HZT ≫ ModuleCat.ofHom Q.HX` (representing $H_X \circ H_Z^T$ in diagrammatic/category-theoretic order) is shown to be zero via `Q.css_condition`.

0.5 Erratum M3: Swapped vertical and horizontal differentials in tensor product double complex

Location. Section II-D ("Double Complexes and Total Complexes"), Definition of the tensor product double complex, `sec_complexes.tex` lines 203–204

Severity. Moderate — The definitions of ∂^v and ∂^h are swapped relative to the paper's own double-complex convention (lines 165–166) and its own figure (lines 208–212); the figure is correct and contradicts the text.

Description. In the definition of the tensor product double complex $C \boxtimes D$, the paper assigns the vertical differential as $\partial^v = \partial^C \otimes \mathrm{id}_D$ and the horizontal differential as $\partial^h = \mathrm{id}_C \otimes \partial^D$. However, by the paper's own convention (lines 165–166), the vertical differential maps $E_{p,q} \to E_{p,q-1}$ (changing the second index q) and the horizontal differential maps $E_{p,q} \to E_{p-1,q}$ (changing the first index p). Since $(C \boxtimes D)_{p,q} = C_p \otimes D_q$, the correct assignment is $\partial^v = \mathrm{id}_C \otimes \partial^D$ (which acts on D_q and maps in the q-direction) and $\partial^h = \partial^C \otimes \mathrm{id}_D$ (which acts on C_p and maps in the p-direction).

Original

$(C \boxtimes D)_{p,q} = C_p \otimes D_q$,
$\partial^v = \partial^C \otimes \mathrm{id}_D$ and $\partial^h = \mathrm{id}_C \otimes \partial^D$.

Corrected

$(C \boxtimes D)_{p,q} = C_p \otimes D_q$,
$\partial^v = \mathrm{id}_C \otimes \partial^D$ and $\partial^h = \partial^C \otimes \mathrm{id}_D$.

Def_11_TensorProductDoubleComplex.lean, lines 45--53

```
/-- The horizontal differential ‘∂^C_p ⊗ id_{D_q} : C_p ⊗ D_q → C_{p'} ⊗ D_q‘.
This maps in the ‘p‘-direction (first index). -/
noncomputable def tensorDh (p p' q : ℤ) : C.X p ⊗ D.X q⟶C.X p' ⊗ D.X q :=
  C.d p p' ▷ D.X q

/-- The vertical differential ‘id_{C_p} ⊗ ∂^D_q : C_p ⊗ D_q → C_p ⊗ D_{q'}‘.
This maps in the ‘q‘-direction (second index). -/
noncomputable def tensorDv (p q q' : ℤ) : C.X p ⊗ D.X q⟶C.X p ⊗ D.X q' :=
  C.X p ◁ D.d q q'
```

Explanation. The paper's double-complex convention (lines 165–166 of `sec_complexes.tex`) defines the vertical differential $\partial^v_{p,q} : E_{p,q} \to E_{p,q-1}$ as the map that decreases the second index q, and the horizontal differential $\partial^h_{p,q} : E_{p,q} \to E_{p-1,q}$ as the map that decreases the first index p. In the tensor product double complex $(C \boxtimes D)_{p,q} = C_p \otimes D_q$, the operator $\mathrm{id}_C \otimes \partial^D$ maps $C_p \otimes D_q \to C_p \otimes D_{q-1}$ (changing only q), so it is the vertical differential; conversely, $\partial^C \otimes \mathrm{id}_D$ maps $C_p \otimes D_q \to C_{p-1} \otimes D_q$ (changing only p), so it is the horizontal differential. The paper's own accompanying commutative diagram (lines 208–212) labels the downward arrows as $\mathrm{id} \otimes \partial^D_q$ and the rightward arrows as $\partial^C_{p-1} \otimes \mathrm{id}$, which is consistent with the corrected assignment and contradicts the text on lines 203–204. The Lean formalization confirms this: `tensorDv` is defined as `C.X p ◁ D.d q q'` (i.e., $\mathrm{id}_{C_p} \otimes \partial^D_q$, the vertical differential mapping in the q-direction), and `tensorDh` is defined as `C.d p p' ▷ D.X q` (i.e., $\partial^C_p \otimes \mathrm{id}_{D_q}$, the horizontal differential mapping in the p-direction). The Lean docstring (lines 87–88) also explicitly documents this correction.

0.6 Erratum M4: Augmentation symbol ϵ conflated with induced map $\bar{\epsilon}$ on $H_0(F)$

Location. Section II-D (sec_complexes.tex), Theorem 4, condition 2, line 345

Severity. Low — notational ambiguity only; the mathematical content is correct, but the same symbol ϵ is used for two distinct maps ($F_0 \to \mathbb{F}_2$ and $H_0(F) \to \mathbb{F}_2$), which may confuse readers

Description. In Theorem 4, condition 2 states that "the augmentation induces an isomorphism $\epsilon : H_0(F) \to \mathbb{F}_2$," reusing the symbol ϵ that was already defined on line 328 as the chain-level augmentation $\epsilon : F_0 \to \mathbb{F}_2$. The map on homology $H_0(F) = F_0/\operatorname{im}(\partial^F) \to \mathbb{F}_2$ is a distinct object induced by ϵ, and should be denoted $\bar{\epsilon}$ to avoid conflation with the chain-level map.

Original

the augmentation induces an isomorphism $\epsilon : H_0(F) \to \mathbb{F}_2$ and

Corrected

the augmentation induces an isomorphism $\bar{\epsilon} : H_0(F) \to \mathbb{F}_2$ and

Thm_4_ProjectionInducesIsomorphism.lean, lines 444--458

```
/-- Under assumptions:
  (1) connection φ acts as identity on homology of F,
  (2) ε⁻ : H₀(F) → 𝔽₂ is an isomorphism (the augmentation ε induces an iso on
      H₀(F)),
  (3) H₀(B) = 0, i.e., range dB = ⊤ (dB is surjective),
  the map H₁(π_*) : H₁(B ⊗_φ F) → H₁(B) = ker(dB) is a linear isomorphism.
  This is the specific isomorphism induced by the chain map π_* from Def_13.
    -/
theorem piStarH1_isLinearEquiv
    (dB : (Fin n₁ → 𝔽₂) →ₗ[𝔽₂] (Fin m₁ → 𝔽₂))
    (dF : (Fin n₂ → 𝔽₂) →ₗ[𝔽₂] (Fin m₂ → 𝔽₂))
    (φ : Connection n₁ m₁ n₂ m₂ dF)
    (ε : (Fin m₂ → 𝔽₂) →ₗ[𝔽₂] 𝔽₂) (hε : ε.comp dF = 0)
    (hφ : ActsAsIdentityOnHomology dB dF φ)
    (hεbar : ((Fin m₂ → 𝔽₂) / range dF) ≃ₗ[𝔽₂] 𝔽₂)
    (hH0B : range dB = ⊤)
    (hε_surj : Function.Surjective ε) :
```

Explanation. The Lean formalization correctly distinguishes the chain-level augmentation from the induced map on homology. In the theorem signature, the chain-level augmentation appears as `ε : (Fin m₂→𝔽₂) →ₗ[𝔽₂] 𝔽₂` (line 454), a linear map on F_0. The condition corresponding to condition 2 of the paper is the separate hypothesis `hεbar : ((Fin m₂→𝔽₂)/ range dF)≃ₗ[𝔽₂] 𝔽₂` (line 456), which asserts that the quotient $F_0/\operatorname{im}(\partial^F)$ — i.e., $H_0(F)$ — is linearly isomorphic to $\mathbb{F}_2$. The docstring (lines 11–13) likewise writes "$\bar{\varepsilon} : H_0(F) \to \mathbb{F}_2$ is an isomorphism" for this hypothesis, using the barred symbol. Thus the formalization treats ϵ and $\bar{\epsilon}$ as two separate objects: ϵ is a linear map on F_0, and $\bar{\epsilon}$ (represented by `hεbar`) is the induced isomorphism on the quotient $H_0(F)$. The paper should adopt the same notational distinction to avoid ambiguity.

0.7 Erratum M5: Strict vs. non-strict inequality in Cheeger constant definition

Location. Section III, `sec_expandercodes.tex`, line 65 (definition of Cheeger constant $h(X)$)

Severity. Low — The strict inequality $|S| < |X_0|/2$ excludes subsets of size exactly $\lfloor |X_0|/2 \rfloor$ from the minimisation, which disagrees with the standard convention $2|S| \leq |X_0|$ used throughout the spectral graph theory literature and in the Lean formalisation. For graphs of even order the two definitions can differ.

Description. The Cheeger constant is defined as a minimum over nonempty vertex subsets S satisfying a size constraint. The paper uses the *strict* inequality $|S| < |X_0|/2$, whereas the standard convention (and the one adopted in the Lean formalisation) is the *non-strict* inequality $2|S| \leq |X_0|$ (equivalently, $|S| \leq |X_0|/2$). When $|X_0|$ is even the strict bound excludes subsets of size exactly $|X_0|/2$, which can change the value of the Cheeger constant.

Original

$$h(X) = \min_{\substack{S \subset V \\ 0 < |S| < \frac{|X_0|}{2}}} \frac{|\delta S|}{|S|}$$

Corrected

$$h(X) = \min_{\substack{S \subset V \\ 0 < |S| \leq \frac{|X_0|}{2}}} \frac{|\delta S|}{|S|}$$

Def_18_CheegerConstant.lean, lines 89--104

```
/-- The eligible subsets for the Cheeger constant: nonempty subsets ‘S ⊆ V‘
with ‘2|S| ≤ |V|‘. The paper uses strict inequality ‘|S| < |V|/2‘ in one place
    ,
but ‘|S| ≤ |V|/2‘ elsewhere; the standard convention is ‘2|S| ≤ |V|‘. -/
def EligibleSubset (S : Finset V) : Prop :=
  S.Nonempty ∧ 2 * S.card ≤ Fintype.card V

/-- The isoperimetric ratio ‘|δS| / |S|‘ for a subset ‘S‘. -/
noncomputable def isoperimetricRatio (G : SimpleGraph V) [DecidableRel G.Adj]
    (S : Finset V) : ℝ :=
  (edgeBoundary G S).card / S.card

/-- The Cheeger constant ‘h(G) = inf_{S} |δS| / |S|‘ where the infimum is over
nonempty subsets ‘S ⊆ V‘ with ‘2|S| ≤ |V|‘. When no eligible subset exists
(e.g., ‘|V| < 2‘), the value is ‘0‘ by the convention ‘sInf ∅ = 0‘ for ‘ℝ‘. -/
noncomputable def cheegerConstant (G : SimpleGraph V) [DecidableRel G.Adj] : ℝ
    :=
  sInf { r : ℝ | ∃ S : Finset V, EligibleSubset S ∧ isoperimetricRatio G S = r
      }
```

Explanation. The Lean formalisation defines an `EligibleSubset` as a nonempty finset S satisfying `2 * S.card ≤ Fintype.card V`, i.e. $2|S| \leq |V|$. The accompanying docstring explicitly flags the discrepancy: "The paper uses strict inequality `|S| < |V|/2` in one place ... the standard convention is `2|S| ≤ |V|`." The non-strict bound $2|S| \leq |X_0|$ is the convention used in standard references (e.g. Hoory–Linial–Wigderson) and is necessary for the discrete Cheeger inequalities $\frac{1}{2}(s - \lambda_2) \leq h(X) \leq \sqrt{2s(s - \lambda_2)}$ to hold as stated. Additionally, Lemma 6 in the paper itself uses $|S| < \alpha|X_0|$ with a generic α, which when specialised to $\alpha = 1/2$ already yields the non-strict bound as the natural boundary case. The paper should therefore replace the strict inequality $|S| < |X_0|/2$ with the non-strict inequality $|S| \leq |X_0|/2$.

0.8 Erratum M6: "Convex" should be "concave" in the proof of edge-to-vertex expansion

Location. Section III-B, Lemma 6 (`lem:edge_to_vertex_expansion`), proof, line 147 of `sec_expandercodes.tex`

Severity. Low — The mathematical argument is correct; only the adjective describing the curvature of $\gamma(\tilde{\alpha})$ is wrong. The linear lower bound used in the proof is valid precisely because the function is concave, so the logical reasoning is sound despite the incorrect label.

Description. In the proof of the edge-to-vertex expansion lemma, the function $\gamma(\tilde{\alpha})$ is described as "convex," but it is in fact concave. A concave function passing through the origin is lower-bounded by any secant line from the origin, which is exactly the property used in the proof; a convex function would be *upper*-bounded by such a line.

Original

Since $\gamma(\tilde{\alpha})$ is a convex function we can lower bound it for $\tilde{\alpha}$ between 0 and α by a linear function with slope $\gamma(\alpha)/\alpha$.

Corrected

Since $\gamma(\tilde{\alpha})$ is a concave function we can lower bound it for $\tilde{\alpha}$ between 0 and α by a linear function with slope $\gamma(\alpha)/\alpha$.

Lem_2_EdgeToVertexExpansion.lean, lines 72--77

```
/-! ## Concavity: gamma(at) * alpha >= gamma(alpha) * at for at <= alpha -/

lemma gammaOfAlpha_concavity (s : ℝ) (lam2 : ℝ) (alpha at_ : ℝ)
    (hs : 0 < s) (hsl : lam2 < s)
    (halpha : 0 < alpha) (hat_nn : 0 ≤ at_) (hat_le : at_ ≤ alpha) :
    gammaOfAlpha s lam2 alpha * at_ ≤ gammaOfAlpha s lam2 at_ * alpha := by
```

Explanation. The Lean formalization makes the concavity property explicit. The section header at line 72 of `Lem_2_EdgeToVertexExpansion.lean` reads `Concavity: gamma(at) * alpha >= gamma(alpha) * at for at <= alpha`, and the lemma `gammaOfAlpha_concavity` (line 74) proves that $\gamma(\alpha) \cdot \tilde{\alpha} \leq \gamma(\tilde{\alpha}) \cdot \alpha$ for $0 \leq \tilde{\alpha} \leq \alpha$, which is equivalent to $\gamma(\tilde{\alpha}) \geq \frac{\gamma(\alpha)}{\alpha}\tilde{\alpha}$. This is the defining inequality for a concave function that passes through the origin: the function lies *above* every secant from the origin. For a convex function the inequality would be reversed, giving an *upper* bound rather than the lower bound claimed in the paper. The main theorem `edgeToVertexExpansion` invokes this concavity at lines 370–372: `gammaOfAlpha_concavity s lam2 alpha at_ hs_pos hlam2_lt_s halpha_pos hat_pos.le hat_le`, confirming that the concavity (not convexity) of γ is the property required. The lemma is fully verified with no `sorry` or `axiom`.

0.9 Erratum M7: Alon–Chung contrapositive requires strict inequality

Location. Section III-B, Corollary 1 (`cor:alon_chung`), `sec_expandercodes.tex`, line 126

Severity. Moderate — The equality condition $|E| = \alpha|X_1|$ is too weak to derive the conclusion by contrapositive; the correct hypothesis is the strict inequality $|E| > \alpha|X_1|$. Downstream uses in the paper (e.g. the Sipser–Spielman distance proof) already treat the condition as a strict inequality, so the final results are unaffected, but the corollary as literally stated does not follow from the Alon–Chung theorem.

Description. Corollary 1 is obtained "by negation" of the Alon–Chung theorem, which states $|X(S)_1| \leq \alpha|X_1|$. The correct contrapositive requires the hypothesis $|E| > \alpha|X_1|$ (strict inequality), but the paper states the condition as $|E| = \alpha|X_1|$ (equality), which does not follow from negating a $\leq$ bound.

Original

Any set of edges $E \subset X_1$ of size $\alpha|X_1|$ is incident to more than $\gamma|X_0|$ vertices.

Corrected

Any set of edges $E \subset X_1$ of size greater than $\alpha|X_1|$ is incident to more than $\gamma|X_0|$ vertices.

Cor_1_AlonChungContrapositive.lean, lines 148--163

```
/-- **Alon-Chung Contrapositive.** If `|E| > α|X₁|`, then `E` is incident to
    more than
`γ|X₀|` vertices. -/
theorem alonChungContrapositive
    (G : SimpleGraph V) [DecidableRel G.Adj]
    (s : ℕ) (hs : 1 ≤ s) (hreg : G.IsRegularOfDegree s)
    (hcard : 2 ≤ Fintype.card V)
    (E : Finset (Sym2 V)) (hE : E ⊆ G.edgeFinset)
    (γ : ℝ) (hγ0 : 0 < γ) (hγ1 : γ < 1)
    (lam₂ : ℝ) (hlam₂ : lam₂ = secondLargestEigenvalue G hcard)
    (hlam₂_nn : 0 ≤ lam₂)
    (hE_large : (E.card : ℝ) > (γ ^ 2 + (lam₂ / ↑s) * γ * (1 - γ)) * G.
        edgeFinset.card) :
    ((incidentVertices E).card : ℝ) > γ * Fintype.card V := by
  by_contra h
  push_neg at h
  have := alonChungContrapositive_direct G s hs hreg hcard E hE γ hγ0 hγ1 lam₂
      hlam₂ hlam₂_nn h
  linarith
```

Explanation. The Alon–Chung theorem (Theorem 1 in the paper) establishes the upper bound $|X(S)_1| \leq \alpha|X_1|$ for any vertex subset S of size $\gamma|X_0|$. Taking the contrapositive yields: if $|E| > \alpha|X_1|$, then the edges in E cannot all lie inside an induced subgraph on $\gamma|X_0|$ vertices, so E must be incident to *more than* $\gamma|X_0|$ vertices. The negation of $\leq$ is $>$, not $=$. The Lean formalisation makes this precise: the hypothesis `hE_large` requires `(E.card : ℝ) > (γ^2 + (lam₂ / s) * γ * (1 - γ)) * G.edge` i.e. a strict inequality. The proof proceeds by contradiction (`by_contra`/`push_neg`), converting the negated conclusion $|\Gamma(E)| \leq \gamma|X_0|$ into the hypothesis of the direct form `alonChungContrapositive_direct`, which yields $|E| \leq \alpha|X_1|$, contradicting the strict inequality assumption via `linarith`.

0.10 Erratum M8: Strict positivity of b required in relative vertex-to-edge expansion

Location. Section III-B, Lemma 3 (`\label{lem:relativevertextoedgeexpansion}`), line 159 of `sec_expandercodes.tex`

Severity. Low — The case $b = 0$ is never used in practice (all applications invoke the lemma with $b = d_L > 0$ or $b = d_{L^\perp} > 0$), but the stated hypothesis technically permits a division-by-zero.

Description. The hypothesis of Lemma 3 states "Let $0 \leq b \leq s$," which admits $b = 0$. However, the conclusion defines $\beta = ((b - \lambda_2) - \alpha(s - \lambda_2))b^{-1}$, requiring division by b; this expression is undefined when $b = 0$. The Lean formalization enforces the strictly positive condition `(hb0 : 0 < b)`, which is also necessary for the final division step in the proof.

Original

Let $S \subset X_0$ be a subset of vertices with $|S| \leq \alpha |X_0|$. Let $0 \leq b \leq s$ and denote

$$A = \{v \in S \mid |(\delta S)_v| \geq s - b\}$$

where for $v \in S$ we define $(\delta S)_v = \delta S \cap \delta v$. Let

$$\beta = ((b - \lambda_2) - \alpha(s - \lambda_2))b^{-1}$$

then there is a lower bound

$$|A| \geq \beta |S|.$$

Corrected

Let $S \subset X_0$ be a subset of vertices with $|S| \leq \alpha |X_0|$. Let $0 < b \leq s$ and denote

$$A = \{v \in S \mid |(\delta S)_v| \geq s - b\}$$

where for $v \in S$ we define $(\delta S)_v = \delta S \cap \delta v$. Let

$$\beta = ((b - \lambda_2) - \alpha(s - \lambda_2))b^{-1}$$

then there is a lower bound

$$|A| \geq \beta |S|.$$

Lem_3_RelativeVertexToEdgeExpansion.lean, lines 227--234

```
theorem relativeVertexToEdgeExpansion (G : SimpleGraph V) [DecidableRel G.Adj]
    (s : ℕ) (α b : ℝ) (hα0 : 0 < α) (hα1 : α < 1) (hb0 : 0 < b) (hbs : b ≤ s)
    (hconn : G.Connected) (hreg : G.IsRegularOfDegree s)
    (hcard : 2 ≤ Fintype.card V)
    (S : Finset V) (hS_le : (S.card : ℝ) ≤ α * Fintype.card V) :
    let lam2 := secondLargestEigenvalue G hcard
    let A := highExpansionVertices G S s b
    ((b - lam2) - α * ((s : ℝ) - lam2)) / b * S.card ≤ A.card := by
```

Explanation. The final step of the proof rearranges the inequality $b\,|A| \geq ((b-\lambda_2)-\alpha(s-\lambda_2))\,|S|$ into $|A| \geq \beta\,|S|$ by dividing both sides by b. In the Lean formalization this is accomplished via `div_le_iff₀ hb0` (line 263), which requires the hypothesis `hb0 : 0 < b` to justify division by a positive real number. When $b = 0$ the expression $\beta = ((b - \lambda_2) - \alpha(s - \lambda_2))/b$ is undefined, and the algebraic rearrangement step is not valid. All downstream applications of the lemma—in Theorem 4 (`thm:expanderviolatedchecks`, with $b = d_L$) and Theorem 5 (`thm:expanderbitdegree`, with $b = d_{L^\perp}$)—use positive code distances, so the correction does not affect any subsequent results.

0.11 Erratum M9: Incorrect use of $d_{L^\perp}$ instead of $d_{L^\perp} - 1$ in the formula for β in Theorem 9

Location. Section III-C ("Expansion properties of Tanner codes"), Theorem 9 (`\label{thm:expanderbitdegree` `sec_expandercodes.tex`, lines 262–268

Severity. Moderate — The formula for β is wrong in both the numerator and denominator; the correct bound is strictly weaker than what the paper claims. Downstream results that plug in this

β inherit the error, though the qualitative conclusions (linear expansion) are unaffected.

Description. In Theorem 9 ("Expander Bit Degree"), the paper writes $d_{L^\perp}$ where it should write $d_{L^\perp} - 1$ in two places in the expression for β. The pigeonhole argument in the proof requires the *strict* inequality $|H^T c_v| \geq d_{L^\perp} > d_{L^\perp} - 1 \geq s - |(\delta S)_v|$, which forces the choice $b = d_{L^\perp} - 1$ in the application of Lemma 3 (Relative Vertex-to-Edge Expansion), not $b = d_{L^\perp}$.

Original

Theorem 1 (Theorem 9)**.** *For $\alpha > 0$ let $y \in C_0(X) \otimes L_0$ such that $|y| \leq \alpha |X_0|(s - k_L)$. Then $|\delta(y)| \geq \beta |y|$ where*

$$\beta = \frac{(d_{L^\perp} - \lambda_2) - \alpha(s - k_L)(s - \lambda_2)}{(s - k_L) d_{L^\perp}}$$

Corrected

Theorem 2 (Theorem 9, corrected)**.** *For $\alpha > 0$ let $y \in C_0(X) \otimes L_0$ such that $|y| \leq \alpha |X_0|(s - k_L)$. Then $|\delta(y)| \geq \beta |y|$ where*

$$\beta = \frac{(d_{L^\perp} - 1 - \lambda_2) - \alpha(s - k_L)(s - \lambda_2)}{(s - k_L)(d_{L^\perp} - 1)}$$

Thm_9_ExpanderBitDegree.lean, lines 402--424

```
    let lam2 := secondLargestEigenvalue G hcard
    let β := ((dLperp - 1 : ℝ) - lam2 - α * m * ((s : ℝ) - lam2)) /
              (m * (dLperp - 1 : ℝ))
    β * totalWeight y ≤ coboundaryWeight G Λ parityCheck y := by
  intro lam2 β
  by_cases hy0 : y = 0
  · subst hy0
    simp only [totalWeight, hammingWeight, Pi.zero_apply, hammingNorm_zero,
      Finset.sum_const_zero, Nat.cast_zero, mul_zero]
    exact Nat.cast_nonneg _
  set S := vertexSupport y
  have hS_pos : 0 < S.card := (vertexSupport_nonempty hy0).card_pos
  have hm_pos : 0 < m := by
    by_contra h; push_neg at h
    have hm0 : m = 0 := Nat.eq_zero_of_le_zero h
    subst hm0; exact hy0 (funext fun v => funext fun i => i.elim0)
  have hS_le : (S.card : ℝ) ≤ α * m * Fintype.card V := by
    calc (S.card : ℝ) ≤ totalWeight y :=
          by exact_mod_cast card_vertexSupport_le_totalWeight y
      _ ≤ α * Fintype.card V * m := hy_weight
      _ = α * m * Fintype.card V := by ring
  set α' := α * m
  set b := (dLperp : ℝ) - 1
```

Explanation. The Lean formalization defines `b := (dLperp : ℝ)- 1` (line 424) and uses this value when invoking the relative vertex-to-edge expansion lemma (line 437: `relativeVertexToEdgeExpansion G s α' b ...`), producing the high-expansion set $A = \{v \in S \mid |(\delta S)_v| \geq s - b = s - (d_{L^\perp} - 1)\}$. The critical step is the pigeonhole argument at lines 534–541: the support of the transpose vector $H^T c_v$ satisfies $|\mathrm{supp}(H^T c_v)| \geq d_{L^\perp}$, while the number of non-boundary label positions satisfies $|\mathrm{nonBdyPos}| \leq b = d_{L^\perp} - 1$. The strict inequality $d_{L^\perp} > d_{L^\perp} - 1$ is what guarantees the existence

of a boundary position j with $(H^T c_v)_j \neq 0$. If one were to set $b = d_{L^\perp}$, then the non-boundary positions could number up to $d_{L^\perp}$, which equals the lower bound on $|\mathrm{supp}(H^T c_v)|$; the strict containment argument would fail, and the pigeonhole step (`h_exists_j` at line 534, which derives a contradiction from suppTV $\subseteq$ nonBdyPos using $d_{L^\perp} \leq$ nonBdyPos.card $\leq b$) would not go through. Hence $d_{L^\perp}$ must be replaced by $d_{L^\perp} - 1$ in both the numerator and denominator of β.

0.12 Erratum M10: Strict vs. non-strict inequality in the Ramanujan bound

Location. Section III-C ("Explicit Constructions of Expander Graphs"), Definition of Ramanujan graphs at line 291 and Theorem 11 (LPS) conclusion at line 347

Severity. Moderate — The standard definition of Ramanujan graphs uses non-strict $\leq$; the paper's use of strict $<$ mischaracterises the optimality condition and is inconsistent with the formalized statement.

Description. The paper defines Ramanujan expanders as graphs satisfying $\lambda_2 < 2\sqrt{s-1}$ (strict inequality) and states the LPS theorem conclusion as $\lambda_2 < 2\sqrt{p}$ (strict inequality). The standard mathematical definition of Ramanujan graphs requires every non-trivial eigenvalue λ to satisfy $|\lambda| \leq 2\sqrt{s-1}$ (non-strict inequality), since these graphs *saturate* the Alon–Boppana bound rather than lying strictly below it. The Lean formalization correctly uses $\leq$.

Original

Line 291: "…the first explicit construction of Ramanujan expanders, which satisfy the optimal bound $\lambda_2 < 2\sqrt{s-1}$."

Lines 341–348, Theorem (LPS), conclusion: "In both cases the graphs $X_{p,q}$ are connected, $p+1$–regular expanders with $\lambda_2 < 2\sqrt{p}$."

Corrected

Line 291: "…the first explicit construction of Ramanujan expanders, which satisfy the optimal bound $\lambda_2 \leq 2\sqrt{s-1}$."

Lines 341–348, Theorem (LPS), conclusion: "In both cases the graphs $X_{p,q}$ are connected, $p+1$–regular expanders with $\lambda_2 \leq 2\sqrt{p}$."

Def_14_GraphExpansion.lean, lines 85--89

```
noncomputable def IsRamanujan (G : SimpleGraph V) [DecidableRel G.Adj]
    (s : ℕ) (hcard : 2 ≤ Fintype.card V) : Prop :=
  ∀ i : Fin (Fintype.card V),
    |adjEigenvalues G i| < (s : ℝ) →
    |adjEigenvalues G i| ≤ 2 * Real.sqrt ((s : ℝ) - 1)
```

Thm_11_LPSRamanujan.lean, lines 386--396

```
theorem lpsGraphPGL_lambda2_le (q : ℕ) [Fact (Nat.Prime q)]
    (p : ℕ) [Fact (Nat.Prime p)] (hpq : p ≠ q)
    (hoddp : p % 2 = 1) (hoddq : q % 2 = 1)
    (hq_gt : (q : ℝ) > 2 * Real.sqrt p)
    (x y : ZMod q) (hxy : x ^ 2 + y ^ 2 + 1 = 0)
    (hram : IsRamanujan (lpsGraphPGL q p hpq hoddp hoddq hq_gt x y hxy) (p +
        1)
      (lpsGraphPGL_card_ge_two q p hpq hoddp hoddq hq_gt))
    (hspec : |secondLargestEigenvalue (lpsGraphPGL q p hpq hoddp hoddq hq_gt x
         y hxy)
      (lpsGraphPGL_card_ge_two q p hpq hoddp hoddq hq_gt)| < (p + 1 : ℝ)) :
    secondLargestEigenvalue (lpsGraphPGL q p hpq hoddp hoddq hq_gt x y hxy)
      (lpsGraphPGL_card_ge_two q p hpq hoddp hoddq hq_gt) ≤ 2 * Real.sqrt p
```

Explanation. The Alon–Boppana theorem (Theorem 10 in the paper) provides an asymptotic *lower* bound: for any family of s-regular graphs with growing size, $\lambda_2 \geq 2\sqrt{s-1} - o(1)$. Ramanujan graphs are defined as those that *meet* this bound, i.e. every non-trivial eigenvalue satisfies $|\lambda| \leq 2\sqrt{s-1}$, not $|\lambda| < 2\sqrt{s-1}$. Indeed, the paper itself says these graphs "saturate this bound" (line 102), which is consistent with $\leq$ but not with strict $<$. The Lean formalization confirms the correct convention: `IsRamanujan` requires `|adjEigenvalues G i| ≤ 2 * Real.sqrt ((s : ℝ) - 1)`, and the concluded spectral bounds `lpsGraphPGL_lambda2_le` and `lpsGraphPSL_lambda2_le` both establish `secondLargestEigenvalue ... ≤ 2 * Real.sqrt p`. The strict inequality in the paper should be replaced with $\leq$ in both the definitional remark at line 291 and in the conclusion of the LPS theorem at line 347.

0.13 Erratum M11: Missing $\frac{1}{|H|}$ normalization in balanced product isomorphism

Location. Section IV-C (Balanced Products of Vector Spaces), `sec_balancedproducts.tex`, line 131

Severity. Moderate — The unnormalized map $\sum_{h\in H} vh \otimes h^{-1}w$ is $|H|$ times a projection rather than a projection; the stated two-sided inverse property fails without the factor $\frac{1}{|H|}$.

Description. The paper states that the isomorphism $V \otimes_H W \xrightarrow{\sim} (V \otimes W)^H$ is given by the map $[v \otimes w] \mapsto \sum_{h\in H} vh \otimes h^{-1}w$. This map is missing the normalization factor $\frac{1}{|H|}$; the correct forward map is the *average* $[v \otimes w] \mapsto \frac{1}{|H|}\sum_{h\in H} vh \otimes h^{-1}w$, which is an idempotent projection onto the invariants.

Original

If H is finite and of odd order, then averaging over all group elements defines an isomorphism

$$V \otimes_H W \xrightarrow{\sim} (V \otimes W)^H,\ [v \otimes w] \mapsto \sum_{h\in H} vh \otimes h^{-1}w$$

Corrected

If H is finite and of odd order, then averaging over all group elements defines an isomorphism

$$V \otimes_H W \xrightarrow{\sim} (V \otimes W)^H,\ [v \otimes w] \mapsto \frac{1}{|H|}\sum_{h\in H} vh \otimes h^{-1}w$$

Def_22_BalancedProductVectorSpaces.lean, lines 95--172

```
variable [Fintype H] [Invertible (Fintype.card H : F2)]

/-- When ‘H‘ is finite of odd order, ‘V tensor_H W = (V tensor W)^H‘.

The forward map sends ‘[v tensor w] |-> (1/|H|) sum_h rho_V(h)(v) tensor rho_W
    (h)(w)‘.
The inverse is the composition of the inclusion and the quotient map. -/
noncomputable def balancedProductInvariantsEquiv :
    balancedProduct rhoV rhoW =L[F2] (rhoV.tprod rhoW).invariants where
  toFun := avgDescended rhoV rhoW
  invFun := invToBalanced rhoV rhoW
  left_inv := by
    intro x
    induction x using Coinvariants.induction_on with
    | h v =>
      ...
      exact averageMap_sub_mem_ker rhoV rhoW v
  right_inv := by
    intro <v, hv>
    ...
    exact averageMap_id _ v hv
```

Explanation. The Lean formalization builds the forward map of the isomorphism `balancedProductInvariantsE` as `avgDescended`, which descends the `averageMap` to the quotient (coinvariants). The `averageMap` is defined via `asAlgebraHom` ρ $(|H|^{-1} \cdot \sum_h$ `of` k H $h)$, i.e. it acts as $x \mapsto \frac{1}{|H|}\sum_{h\in H}\rho(h)(x)$. The hypothesis `[Invertible (Fintype.card H :` $\mathbb{F}_2$`)]` (line 99) is required precisely to make sense of $\frac{1}{|H|}$ in the base field; this constraint forces $|H|$ to be odd (since $\operatorname{char}\mathbb{F}_2 = 2$). With the $\frac{1}{|H|}$ factor, the averaging map is idempotent: `averageMap_id` (used on line 170) shows that it acts as the identity on invariant elements, establishing the right-inverse property. Without the normalization, the sum $\sum_{h\in H}\rho(h)(x)$ equals $|H|$ times the projection on invariants, so the round-trip via inclusion followed by the unnormalized sum would multiply every invariant element by $|H|$, which is not the identity unless $|H| = 1$.

0.14 Erratum M12: Swapped left/right H-action convention in Künneth balanced product lemma

Location. Section III-D, Lemma 1 (`lem:kuennethbalanced`), line 175 of `sec_balancedproducts.tex`

Severity. Moderate — The left/right assignment of the H-action on C and D is reversed relative to the standard convention required for the balanced product $C \otimes_H D$ to be well-defined; however, the mathematical content of the lemma (the isomorphism itself) is correct under the proper convention.

Description. Lemma 1 states that H acts on C from the *left* and on D from the *right*. The balanced product $C \otimes_H D = (C \otimes D)/\langle vh \otimes w - v \otimes hw\rangle$ requires C to carry a *right* H-action and D to carry a *left* H-action so that the identification $c \cdot h \otimes d \sim c \otimes h \cdot d$ is well-formed. The paper's own definition of the balanced product of vector spaces (Section III-C, line 118) correctly states "right, respectively left," but the lemma reverses this assignment.

Original

Let H be a finite group of odd order acting on C and D from the left and right, respectively. There is an isomorphism

$$H_n(C \otimes_H D) \cong \bigoplus_{p+q=n} H_p(C) \otimes_H H_q(D).$$

Corrected

Let H be a finite group of odd order acting on C and D from the right and left, respectively. There is an isomorphism

$$H_n(C \otimes_H D) \cong \bigoplus_{p+q=n} H_p(C) \otimes_H H_q(D).$$

Lem_4_KunnethBalancedProduct.lean, lines 12--16

```
Let ‘H‘ be a finite group of odd order, and let ‘C‘, ‘D‘ be ‘H‘-equivariant
    chain complexes
over ‘F_2‘. Then there is an isomorphism
$$H_n(C \otimes_H D) \cong \bigoplus_{p+q=n} H_p(C) \otimes_H H_q(D)$$
```

Def_23_BalancedProductChainComplex.lean, lines 8--10

```
Given a group ‘H‘, chain complexes ‘C‘ and ‘D‘ with ‘H‘-equivariant
    differentials (where each
‘C_i‘ carries a right ‘H‘-action and each ‘D_j‘ carries a left ‘H‘-action),
    the **balanced
product double complex** ‘C (*\boxtimes$_H$*) D‘ has objects ‘(C (*\
    boxtimes$_H$*) D)_{p,q} = C_p (*\otimes$_H$*) D_q‘
```

Def_22_BalancedProductVectorSpaces.lean, lines 56--58

```
/-- The paper's balancing relation: ‘[rho_V(h^{-1})(v) (*$\otimes$*) w] = [v
    (*$\otimes$*) rho_W(h)(w)]‘.
In right-action notation, this says ‘[vh (*$\otimes$*) w] = [v (*$\otimes$*)
    hw]‘.
Proof: apply diagonal relation at ‘h‘ to ‘(rho_V(h^{-1})(v), w)‘. -/
```

Explanation. The balanced product $V \otimes_H W$ is defined (Section III-C of the paper, line 118–119) as the quotient $V \otimes W / \langle vh \otimes w - v \otimes hw \rangle$, which requires the first factor to carry a right H-action (so that vh is meaningful) and the second factor to carry a left H-action (so that hw is meaningful). The Lean formalization encodes this convention consistently throughout: `Def_23` declares that "each `C_i` carries a right `H`-action and each `D_j` carries a left `H`-action" (lines 8–10), and `Def_22` establishes the balancing relation $[\rho_V(h^{-1})(v) \otimes w] = [v \otimes \rho_W(h)(w)]$, i.e. $[v \cdot h \otimes w] = [v \otimes h \cdot w]$ in right-action notation (lines 56–58). The axiom signature `kunnethBalancedProduct` at line 233 of the Lean file takes `C D : HEquivariantChainComplex H`, inheriting this right-on-C, left-on-D convention. Lemma 1 in the paper reverses the assignment to "left and right, respectively," which contradicts both the paper's own earlier definition and the Lean formalization. The fix is to swap "left" and "right" so that H acts on C from the right and on D from the left.

0.15 Erratum M13: Incorrect orbit membership in quotient graph adjacency definition

Location. Section III-B, "Quotient graph and trivialisation," line 72 of `sec_balancedproducts.tex`

Severity. Moderate — The typo silently collapses adjacency between two *distinct* vertex orbits into a self-adjacency condition on a single orbit, which contradicts the intended definition of the quotient graph.

Description. In the definition of the quotient graph X/H, the paper states the adjacency condition for two vertex orbits $\mathcal{V}$ and $\mathcal{V}'$ but erroneously writes $v' \in \mathcal{V}$ instead of $v' \in \mathcal{V}'$ for the second vertex. This makes both vertices belong to the same orbit, which does not correctly define adjacency between distinct orbits in the quotient.

Original

Hence, two vertices $\mathcal{V}$, $\mathcal{V}'$ are adjacent in X/H if there are $v \in \mathcal{V}$ and $v' \in \mathcal{V}$ such that v and v' are adjacent in X.

Corrected

Hence, two vertices $\mathcal{V}$, $\mathcal{V}'$ are adjacent in X/H if there are $v \in \mathcal{V}$ and $v' \in \mathcal{V}'$ such that v and v' are adjacent in X.

`Def_24_QuotientGraphTrivialization.lean, lines 111--138`

```
def quotientBdry (E : X.quotientEdges) : Finset X.quotientVertices :=
  Quotient.liftOn E
    (fun e => (X.graph.bdry e).image X.vertexQuot)
    (fun e₁ e₂ (h : @Setoid.r _ (MulAction.orbitRel H (X.graph.cells 1)) e₁ e₂
        ) => by
      obtain ⟨g, hg⟩ := h
      ext V
      simp only [Finset.mem_image]
      constructor
      · rintro ⟨τ, hτ, rfl⟩
        rw [← hg] at hτ
        have hτ' : g⁻¹ • τ ∈ X.graph.bdry e₂ := by
          rwa [← X.bdry_equivariant e₂ g τ]
        exact ⟨g⁻¹ • τ, hτ', by
          apply Quotient.sound
          exact ⟨g⁻¹, rfl⟩⟩
      · rintro ⟨τ, hτ, rfl⟩
        have hτ' : g • τ ∈ X.graph.bdry e₁ := by
          rw [← hg]
          exact X.bdry_orbit_compat e₂ g τ hτ
        exact ⟨g • τ, hτ', by
          apply Quotient.sound
          exact ⟨g, rfl⟩⟩)
```

Explanation. The Lean formalization correctly captures the intended semantics. The function `quotientBdry E` maps an edge orbit E to the finset of vertex orbit classes $\{$`X.vertexQuot` $\tau \mid \tau \in \partial(\mathrm{repr}(E))\}$, producing one quotient vertex for each boundary endpoint of a representative edge. Because a graph edge connects two vertices that generally lie in *different* orbits, the resulting finset contains distinct vertex orbits $\mathcal{V}$ and $\mathcal{V}'$. This distinctness is explicitly enforced throughout the formalization: the `connectionOnEdge` definition (line 202) and the `inducedConnection_spec` theorem

(line 269) both carry the hypothesis `hne : X.vertexQuot` $\tau_1 \neq$ `X.vertexQuot` τ_2, which requires the two boundary vertices to belong to *different* orbits. The paper's quotient condition (no edge connects v to vh for $h \neq 1$) guarantees this distinctness, and the Lean code faithfully models it. The paper's typo—writing $v' \in \mathcal{V}$ instead of $v' \in \mathcal{V}'$—is therefore inconsistent with both the formal definition and the surrounding text that treats $\mathcal{V}$ and $\mathcal{V}'$ as two separate orbits.

0.16 Erratum M14: Incorrect index range for cycle graph chain groups

Location. Section IV-D ("Balanced Product with a Circle"), `sec_balancedproducts.tex`, lines 335 and 340

Severity. Moderate — The cycle graph C_ℓ is a 1-complex with chain groups at degrees 0 (vertices) and 1 (edges); writing "$i = 1, 2$" references a non-existent degree-2 group and omits degree 0, which could mislead readers about the structure of the balanced product double complex.

Description. The paper states that the maps $\mathbb{F}_2[\mathbb{Z}_\ell] \to C_i(C_\ell)$ and $V \to V \otimes_{\mathbb{Z}_\ell} C_i(C_\ell)$ are isomorphisms "for $i = 1, 2$" in two consecutive paragraphs. Since C_ℓ is a 1-dimensional complex whose chain groups are $C_0(C_\ell)$ (vertices) and $C_1(C_\ell)$ (edges), and the paper itself fixes y_0 as a vertex and y_1 as an edge, the correct range is $i = 0, 1$.

Original

Fix some vertex y_0 and edge y_1 in C_ℓ. Then the maps

$$\mathbb{F}_2[\mathbb{Z}_\ell] \to C_i(C_\ell), \sum_h a_h h \mapsto \sum_h a_h y_i h$$

are isomorphisms for $i = 1, 2$. Similarly, for every vector space V with a linear right action the maps

$$V \to V \otimes_{\mathbb{Z}_\ell} C_i(C_\ell), v \mapsto [v \otimes y_i].$$

are isomorphism for $i = 1, 2$.

Corrected

Fix some vertex y_0 and edge y_1 in C_ℓ. Then the maps

$$\mathbb{F}_2[\mathbb{Z}_\ell] \to C_i(C_\ell), \sum_h a_h h \mapsto \sum_h a_h y_i h$$

are isomorphisms for $i = 0, 1$. Similarly, for every vector space V with a linear right action the maps

$$V \to V \otimes_{\mathbb{Z}_\ell} C_i(C_\ell), v \mapsto [v \otimes y_i].$$

are isomorphism for $i = 0, 1$.

Def_26_BalancedProductTannerCycleCode.lean, lines 160--164

```
private def cycleObj : ℤ → ModuleCat 𝔽₂
  | (0 : ℤ) => ModuleCat.of 𝔽₂ (Fin ℓ → 𝔽₂)
  | (1 : ℤ) => ModuleCat.of 𝔽₂ (Fin ℓ → 𝔽₂)
  | (Int.ofNat (_ + 2)) => ModuleCat.of 𝔽₂ PUnit
  | (Int.negSucc _) => ModuleCat.of 𝔽₂ PUnit
```

Explanation. The Lean formalization defines `cycleObj` (lines 160–164) with non-trivial modules (`Fin ℓ → 𝔽₂`) at integer degrees 0 and 1, and trivial modules (`PUnit`) at all other degrees. The sole non-trivial differential `cycleMor` maps degree 1 to degree 0 (line 167: `| 1, 0 => ModuleCat.ofHom (CycleGraph.differential ℓ)`), confirming the cycle complex is concentrated in degrees $\{0, 1\}$. Furthermore, the inclusion maps into the total complex at degree 1 use `bpObj ... 1 0` and `bpObj ... 0 1` (lines 309–318), reflecting that the two summands of C_1^{Tot} are $C_1(X, L) \otimes_{\mathbb{Z}_\ell} C_0(C_\ell)$ and $C_0(X, L) \otimes_{\mathbb{Z}_\ell} C_1(C_\ell)$—both involving cycle-complex degrees 0 and 1, not 1 and 2. The paper's "$i = 1, 2$" is therefore an off-by-one indexing error; the correct statement is "$i = 0, 1$."

Concordance Table

Summary of all errata with Lean file references

ID	Severity	Paper Location	Summary
S1	High	Sec. III-C, Thm. 8	Incorrect α', b, and β'': should use $\alpha' = \alpha s$, $b = d_L - 1$, yielding corrected β'' (`Thm_8_ExpanderViolatedChecks.lean`)
S2	Low	Sec. IV-F, Cor. 23	Missing β_{co} in D_X bound, asymptotically irrelevant (`Thm_15_ExplicitFamilyQuantumCodes.lean`)
M1	Moderate	Sec. II-A, line 32	Coboundary index: $(\partial_{i-1})^{tr}$ should be $(\partial_{i+1})^{tr}$ (`Rem_2_NotationConventions.lean`)
M2	Moderate	Sec. II-B, line 85	CSS condition: $H_Z^T H_X = 0$ should be $H_X H_Z^T = 0$ (`Def_4_CSSCode.lean`)
M3	Moderate	Sec. II-D, ll. 203–204	Swapped ∂^v and ∂^h in tensor product double complex (`Def_11_TensorProductDoubleComplex.lean`)
M4	Low	Sec. II-D, Thm. 4	Augmentation ϵ conflated with induced map $\bar{\epsilon}$ on $H_0(F)$ (`Thm_4_ProjectionInducesIsomorphism.lean`)
M5	Low	Sec. III, line 65	Cheeger constant: strict $<$ should be $\leq$ (`Def_18_CheegerConstant.lean`)
M6	Low	Sec. III-B, Lem. 6	"Convex" should be "concave" for $\gamma(\tilde{\alpha})$ (`Lem_2_EdgeToVertexExpansion.lean`)
M7	Moderate	Sec. III-B, Cor. 1	Contrapositive requires strict $\lvert E\rvert > \alpha\lvert X_1\rvert$, not equality (`Cor_1_AlonChungContrapositive.lean`)
M8	Low	Sec. III-B, Lem. 3	Hypothesis $0 \leq b$ should be $0 < b$, avoids division by zero (`Lem_3_RelativeVertexToEdgeExpansion.lean`)
M9	Moderate	Sec. III-C, Thm. 9	$d_{L^\perp}$ should be $d_{L^\perp} - 1$ in β formula (`Thm_9_ExpanderBitDegree.lean`)
M10	Moderate	Sec. III-C, Thm. 11	Ramanujan bound: strict $<$ should be $\leq$ (`Def_14_GraphExpansion.lean`)
M11	Moderate	Sec. IV-C, line 131	Missing $\frac{1}{\lvert H\rvert}$ normalization in balanced product map (`Def_22_BalancedProductVectorSpaces.lean`)
M12	Moderate	Sec. III-D, Lem. 1	Left/right H-action on C, D swapped (`Lem_4_KunnethBalancedProduct.lean`)
M13	Moderate	Sec. III-B, line 72	Orbit membership: $v' \in \mathcal{V}$ should be $v' \in \mathcal{V}'$ (`Def_24_QuotientGraphTrivialization.lean`)
M14	Moderate	Sec. IV-D, ll. 335, 340	Index range $i = 1, 2$ should be $i = 0, 1$ for cycle graph (`Def_26_BalancedProductTannerCycleCode.lean`)